

Day 7 — Cloud Storage Solutions

Amazon S3, EBS, EFS & Glacier

Today we break down the four main pillars of AWS storage infrastructure. By the end of this session, you will know exactly when to use a high-performance virtual drive versus when to build an automated archiving lifecycle policy — and how to make storage decisions that balance performance with corporate budget reality.

COURSEWARE MODULE · DAY 7



Today's Learning Objectives

By the end of Day 7, you will be able to confidently answer these core operational questions:

01

Understand the Storage Spectrum

Distinguish between Block, File, and Object storage models and identify the AWS service that implements each.

03

Control Cloud Costs

Design S3 Lifecycle Policies to automate data movement and prevent runaway cloud storage invoices.

02

Align Storage to Use Cases

Match the correct storage tier to real-world vocational scenarios — from OS volumes to compliance archives.

04

Avoid Career-Ending Mistakes

Recognize anti-patterns like manual file migration scripts that compromise data integrity and trigger compliance audits.

Lecture Outline

Here is the roadmap for Day 7. We will move through theory, live walkthroughs, analogies, and a real-world case study.

1

The Storage Taxonomy

Block vs. Object vs. File — core concepts and structural differences.

2

Amazon S3 Deep Dive

Buckets, keys, metadata, durability, and automated storage class tiers.

3

EBS & EFS

Block volumes for single instances vs. shared network file systems for compute fleets.

4

Case Study

MedVanguard Health Systems — a real compliance and cost crisis, and how to fix it.



The Cloud Storage Spectrum

AWS offers three fundamentally different models of storing data. Each one reflects a distinct way data is physically organized, accessed, and billed. Choosing the wrong model can crash an application or cost your organization thousands in unnecessary fees.

Block Storage

Data split into raw, addressable sectors. Reads and writes happen at the byte level — like a local hard drive.

File Storage

Hierarchical directory structure accessible over a network. Multiple clients share the same file system concurrently.

Object Storage

Flat structure where each file is a discrete object with a unique key, raw data payload, and rich metadata tags.

Block Storage vs. Object Storage: Core Taxonomy

Block Storage

Data is divided into fixed-size sector blocks at the hardware level. When you modify a single character in a 1 GB file, only the specific sector block containing that character is updated — not the entire file.

- Extremely low latency, high throughput
- Behaves like a local hard drive or SSD
- Required for operating systems and databases
- AWS implementation: **Amazon EBS**

Object Storage

Files are stored as complete, indivisible entities called "objects." Changing a single character requires downloading the entire object, modifying it locally, and re-uploading the complete file payload.

- Infinite horizontal scalability
- Each object carries rich custom metadata
- Globally accessible via HTTPS endpoints
- AWS implementation: **Amazon S3**

 The tradeoff is clear: Block storage wins on performance and mutability. Object storage wins on scale, cost, and global reach.

Why the Distinction Matters Operationally



Block Storage Use Cases

Use block storage whenever your workload requires **random read/write access** at the file system level. This includes: running a relational database like MySQL or PostgreSQL, hosting a virtual machine operating system, or processing financial transaction logs that require millisecond latency.



Object Storage Use Cases

Use object storage for **static assets that are written once and read many times**. This includes: hosting website images and videos, storing log files and analytics exports, archiving compliance documents, and distributing software packages — any scenario where infinite scale matters more than sub-millisecond write performance.

Introduction to Amazon S3

Simple Storage Service — The Foundation of AWS Data Infrastructure

Amazon S3 is one of the oldest and most powerful services in the AWS ecosystem, launched in 2006. It provides a flat, infinitely scalable object storage layer accessible via standard HTTPS endpoints from anywhere on the internet. Every enterprise-grade AWS architecture uses S3 in some capacity.



Anatomy of an S3 Object

Every file stored in Amazon S3 is treated as a discrete **object**. Each object is composed of exactly three structural components. Understanding this anatomy is critical for working with S3 programmatically and via the console.

1

Key

The unique string identifier for the object – functionally equivalent to a full file path. Example:

`reports/2024/jan/billing-summary.pdf`. The key is how you locate and retrieve any object in a bucket.

2

Value

The actual raw binary data payload of the object – the file contents itself. This can be anything from a 1 KB text file to a 5 TB video archive. S3 supports individual object sizes up to 5 terabytes.

3

Metadata

A collection of key-value pairs that describe the object. Includes system metadata (content type, upload date, file size) and custom user-defined tags (department, project code, data classification level).

S3 Buckets & Durability

What Is a Bucket?

An S3 Bucket is a globally unique container that holds your objects. Bucket names must be unique across *all* AWS accounts worldwide. Think of it as the root namespace for your storage — all objects live inside a bucket, addressed by their key string.

- Unlimited objects per bucket
- Region-specific but globally accessible
- Fine-grained access control via bucket policies and IAM
- Versioning supported — retain all historical versions of objects

11 Nines of Durability

Amazon S3 is designed for **99.999999999% durability**. This means if you store 10 million files in S3, you might statistically lose *one file every 10,000 years*. AWS achieves this by automatically replicating every object across a minimum of three physically separated Availability Zones within the selected region. This structural reliability is impossible to replicate in a standard on-premises server closet — and it fundamentally changes corporate liability calculations.

S3 Storage Classes Overview

Not all data is accessed equally. AWS provides distinct storage tiers that let you match your access patterns to your spending. Storing everything in S3 Standard is one of the most common and costly mistakes in cloud architecture.



The staircase represents a deliberate design principle: as access frequency decreases, storage cost drops – but retrieval latency increases. Your Lifecycle Policy automates the movement between tiers.

S3 Standard — The Active Data Tier

When to Use S3 Standard

S3 Standard is the default storage class for any object uploaded to Amazon S3. It is engineered for workloads that require **frequent, low-latency access** to data — typically accessed more than once per month. It delivers the highest throughput and the lowest retrieval latency of all S3 tiers.

- Active web application assets (images, CSS, JS)
- Real-time analytics data pipelines
- Frequently accessed database backup snapshots
- Mobile application content delivery

Cost Profile

S3 Standard carries the **highest per-GB monthly storage fee** among all S3 tiers, but there are **no retrieval fees** — you pay only for storage and data transfer out. This makes it cost-effective when data is accessed constantly, but extremely wasteful when data ages and sits idle for months or years.



Leaving compliance archives or rarely accessed historical logs in S3 Standard is a top cause of cloud budget overruns. Always implement a Lifecycle Policy.



S3 Standard-Infrequent Access (S3-IA)

S3 Standard-IA is designed for data that must remain **immediately retrievable** (millisecond latency) but is accessed less than once per month. It offers a significantly lower monthly storage fee than S3 Standard, in exchange for a per-GB **retrieval fee** each time data is read. This tradeoff only saves money when retrieval events are infrequent.

Best For

Disaster recovery backups, monthly financial reports, secondary database replicas, and any file you need available instantly but rarely touch.

Avoid When

Data is accessed daily or multiple times per week – the retrieval fees will quickly eclipse the storage savings, making S3 Standard the cheaper option.

Amazon S3 Glacier & Glacier Deep Archive

The Glacier family represents AWS's long-term digital vault tier — engineered for compliance archives and data that may never be accessed again, but must be preserved with structural reliability for years or decades.



Glacier Instant Retrieval

Lowest cost for archive data with **millisecond retrieval**. Use for medical imaging or news media that is accessed quarterly but needs immediate delivery when requested.



Glacier Flexible Retrieval

Retrieval time ranges from **1–5 minutes** (Expedited) to **3–5 hours** (Standard). Very low cost. Ideal for backup and disaster recovery archives where time is not critical.



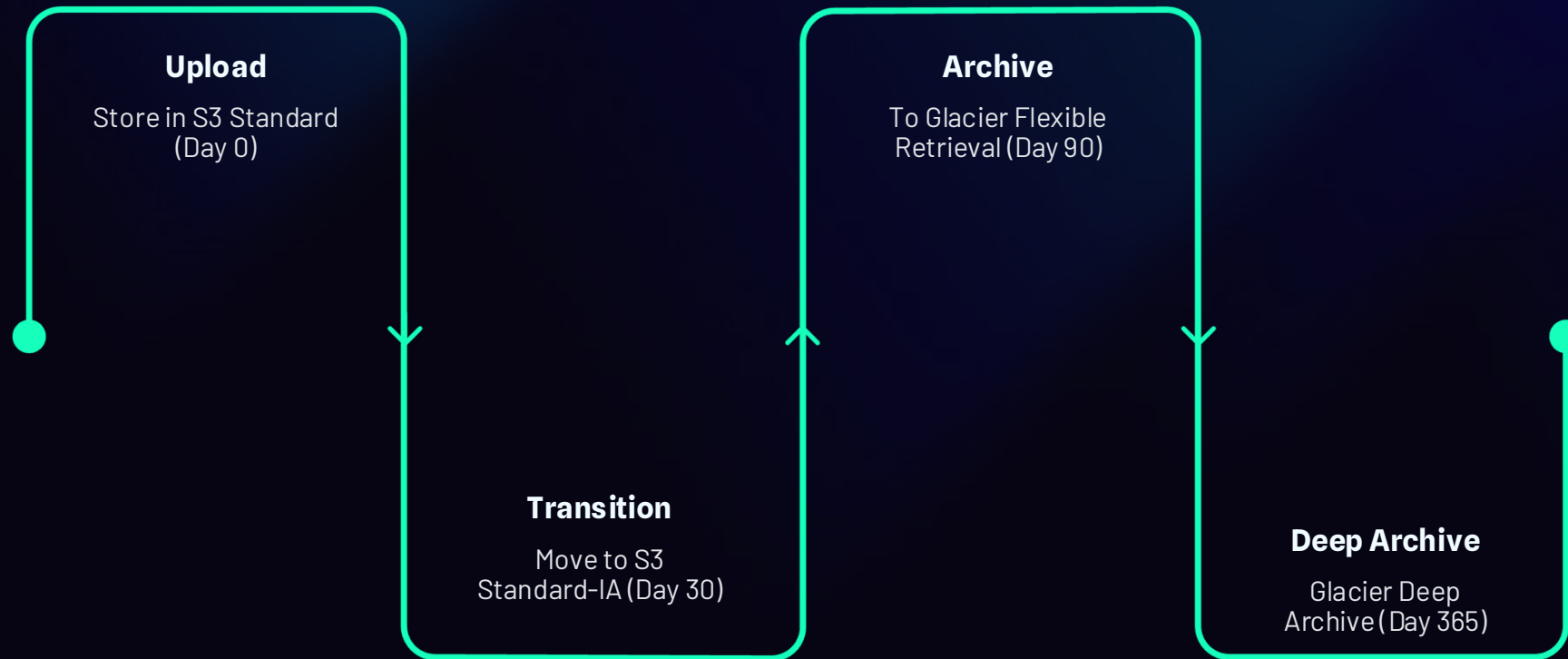
Glacier Deep Archive

The lowest cost storage in AWS — fractions of a cent per GB/month. Retrieval takes up to **12 hours**. Designed for regulatory compliance data retained 7–10+ years with very rare access.

S3 Lifecycle Policies

Automating Cost Optimization — No Manual Intervention Required

An S3 Lifecycle Policy is a programmatic rule set — defined via the AWS Console or JSON configuration — that automatically transitions objects between storage classes based on their age. This eliminates the need for administrators to manually track and migrate files, removing human error from cost optimization entirely.



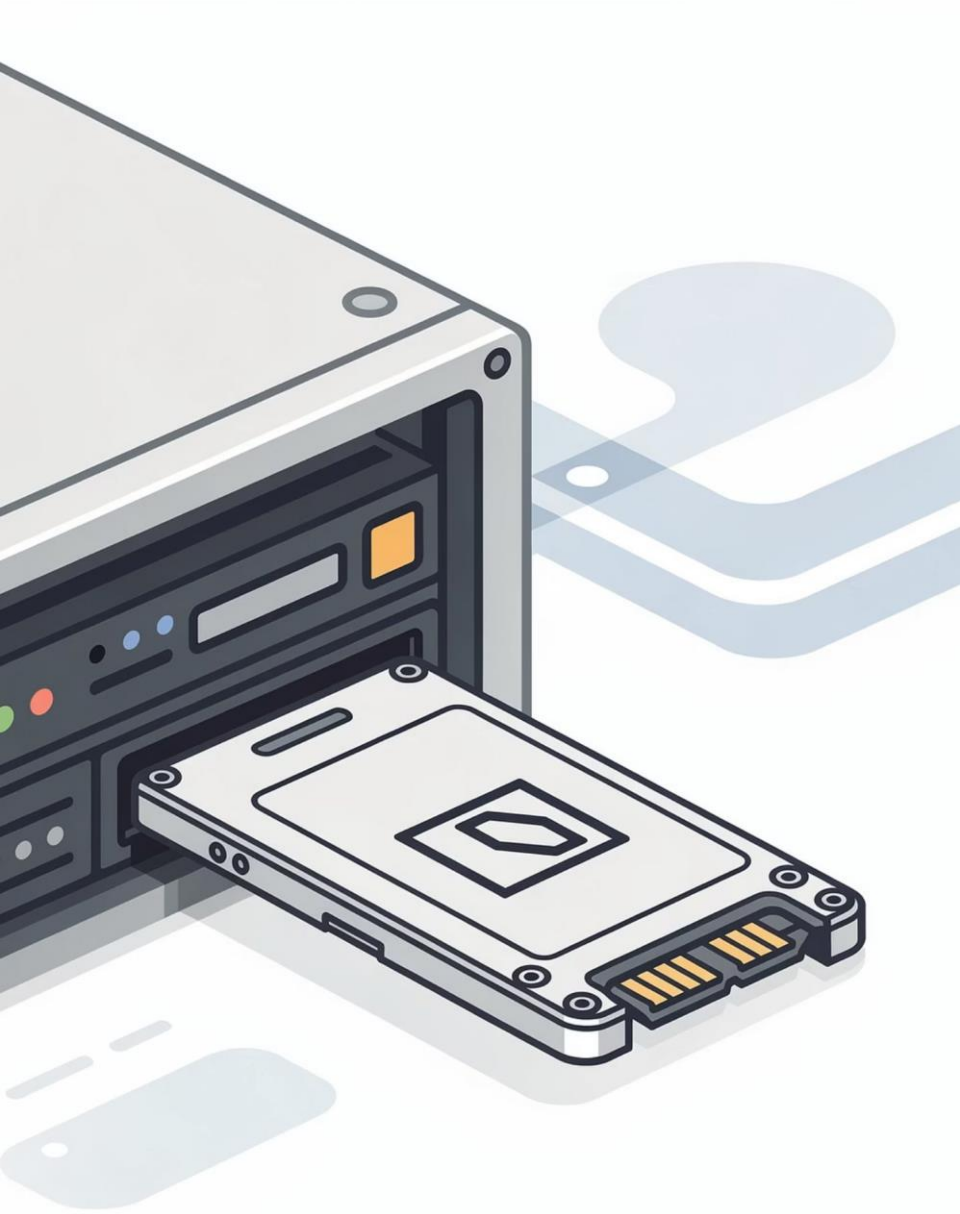
A single Lifecycle Policy replaces months of manual audit work. Once configured, the policy continuously enforces your data tier strategy at scale — across petabytes — without any administrator intervention.

Configuring a Lifecycle Policy — Example Rule

Below is a representative S3 Lifecycle Policy structure for a medical records compliance scenario. This rule is applied at the bucket or prefix level and executes automatically on a nightly AWS evaluation cycle.

```
{
  "Rules": [
    {
      "ID": "MedicalImagingArchivePolicy",
      "Status": "Enabled",
      "Filter": { "Prefix": "patient-records/" },
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "STANDARD_IA"
        },
        {
          "Days": 90,
          "StorageClass": "GLACIER"
        },
        {
          "Days": 365,
          "StorageClass": "DEEP_ARCHIVE"
        }
      ],
      "Expiration": { "Days": 2555 }
    }
  ]
}
```

The `Filter.Prefix` field scopes the policy to a **specific** folder path. The `Expiration` field permanently deletes objects after 7 years (2,555 days), satisfying the legal maximum retention window automatically.



Amazon Elastic Block Store (EBS)

High-Performance Block Storage for EC2 Instances

Amazon EBS is the network-attached block storage service for EC2 virtual servers. It functions exactly like an external SSD plugged directly into a single machine — providing raw, sector-level storage that an operating system can format, partition, and use as a local hard drive. EBS volumes persist independently from the EC2 instance lifecycle, surviving reboots and instance terminations.

EBS: Core Characteristics

Single-Instance Attachment

A standard EBS volume can be attached to **only one EC2 instance at a time** within the same Availability Zone. This isolation provides maximum performance but prevents concurrent multi-server access. (Exception: EBS Multi-Attach is available for io1/io2 volumes in specific scenarios.)

Availability Zone Locked

An EBS volume exists within a specific Availability Zone. An EC2 instance in `us-east-1a` cannot directly attach an EBS volume created in `us-east-1b`. To migrate, you must create an EBS Snapshot and restore it in the target zone.

Persistent & Independent Lifecycle

Unlike ephemeral instance store volumes (which are destroyed when an instance stops), EBS data persists. You can detach a volume from one instance, stop it, and re-attach it to a replacement instance – with all data intact.

EBS Volume Types

AWS offers multiple EBS volume types: **gp3** (General Purpose SSD – recommended default), **io2** (Provisioned IOPS for high-performance databases), **st1** (Throughput-optimized HDD for sequential workloads), and **sc1** (Cold HDD for infrequently accessed data).

EBS Snapshots: Backup & Migration

What Is an EBS Snapshot?

An EBS Snapshot is a point-in-time backup of an EBS volume, stored durably in Amazon S3 (managed by AWS). Snapshots are **incremental** — after the first full snapshot, subsequent snapshots only capture the blocks that changed, reducing storage costs and backup time.

- Create snapshots manually or via AWS Backup schedules
- Restore a snapshot to a new volume in any AZ or region
- Share snapshots across AWS accounts
- Encrypt snapshots with AWS KMS keys

Common Snapshot Use Cases

EBS Snapshots are the primary mechanism for several critical operational workflows:

- **Disaster Recovery:** Restore a failed server's OS volume from the latest snapshot
- **Cross-Region Migration:** Copy a snapshot to another region to launch an identical server
- **Golden AMI Builds:** Create a standardized machine image from a configured volume
- **Pre-Change Checkpoints:** Snapshot before a risky system update for instant rollback capability



Amazon Elastic File System (EFS)

Fully Managed Shared Network File Storage

Amazon EFS is a fully managed, scalable, shared file system that uses the NFS (Network File System) protocol. Unlike EBS, EFS breaks the single-instance isolation barrier — **thousands of EC2 instances across multiple Availability Zones can simultaneously read and write to the same EFS file system**. As data grows, EFS automatically expands to petabyte scale with zero provisioning required.

EFS vs. EBS: Head-to-Head

Attribute	Amazon EBS	Amazon EFS
Storage Type	Block Storage	Network File System (NFS)
Concurrent Access	1 instance at a time (standard)	Thousands of instances simultaneously
Scope	Single Availability Zone	Multi-AZ by default
Scaling	Manual provisioning required	Automatic, elastic scaling to petabytes
Pricing Model	Pay for provisioned capacity	Pay only for data stored (no over-provisioning)
Primary Use Case	OS volumes, databases, single-server apps	Shared web content, CMS, container storage
Protocol	Block I/O (OS managed)	NFSv4.1 / NFSv4.0

When to Choose EFS Over EBS

The defining question is: **Do multiple servers need simultaneous access to the same data?** If yes, EFS is the correct choice. EBS is not designed for this scenario and will cause data corruption in shared-write configurations.



Auto-Scaling Web Fleets

When 10+ web servers behind a load balancer all need to serve the same uploaded user images or application files, EFS provides a single consistent source of truth for all nodes simultaneously.



Container Workloads

Docker containers on ECS or EKS frequently need shared persistent storage. EFS integrates natively with both services and is the recommended persistent volume option for containerized applications.



CMS & Shared Applications

Content Management Systems like WordPress in multi-server configurations require all instances to access a shared `/wp-content/uploads` directory. EFS mounts identically on every server in the fleet.

CHAPTER BREAK

Real-World Analogies

Before we move into the case study, let's anchor these technical concepts with two practical analogies from the physical world. Great system administrators think in mental models — and the best mental models are ones you can see.



Analogy 1: The Commercial Airport

Mapping AWS Storage Services to Airport Operations

Managing AWS storage architecture is remarkably similar to running a major international airport's luggage and maintenance operations. Each storage service maps directly to a physical airport system — making the abstract tangible and memorable for your career troubleshooting work.

Airport Analogy: EBS — The Mechanic's Toolbox

The Physical Scenario

An EBS volume is like a heavy-duty rolling mechanic's toolbox locked inside a specific aircraft hangar. It contains specialized instruments for performing core diagnostics on that one aircraft.

- The toolbox must be **physically latched to one specific airplane** — it cannot be shared across the tarmac with another plane simultaneously
- It operates at maximum efficiency for that single aircraft
- If the plane is retired, the **toolbox remains safely in the hangar**, ready to be transferred to the next aircraft

The AWS Mapping

Just like the toolbox, an EBS volume provides **maximum performance for a single attached EC2 instance**. It cannot be dynamically shared with another instance at the same time. And just like the toolbox surviving a plane's retirement, an EBS volume **persists independently** from an EC2 instance lifecycle. Terminate the instance, the volume remains — ready to be re-attached to a replacement server.

 This "one toolbox, one plane" constraint is the most important operational limitation to internalize about EBS.

Airport Analogy: S3 — The Luggage Repository

An S3 bucket is like a massive, infinite central luggage processing facility. When a passenger drops off a suitcase, the airport treats the entire bag as a single closed package — an **Object**. A barcode sticker is attached with the passenger's name, flight, and bag weight — the **Metadata**. The automated system retrieves the bag using that specific barcode sequence — the **Key**.

Barcode = Key

The unique tracking string used to locate and retrieve any object in the system.

Suitcase = Value

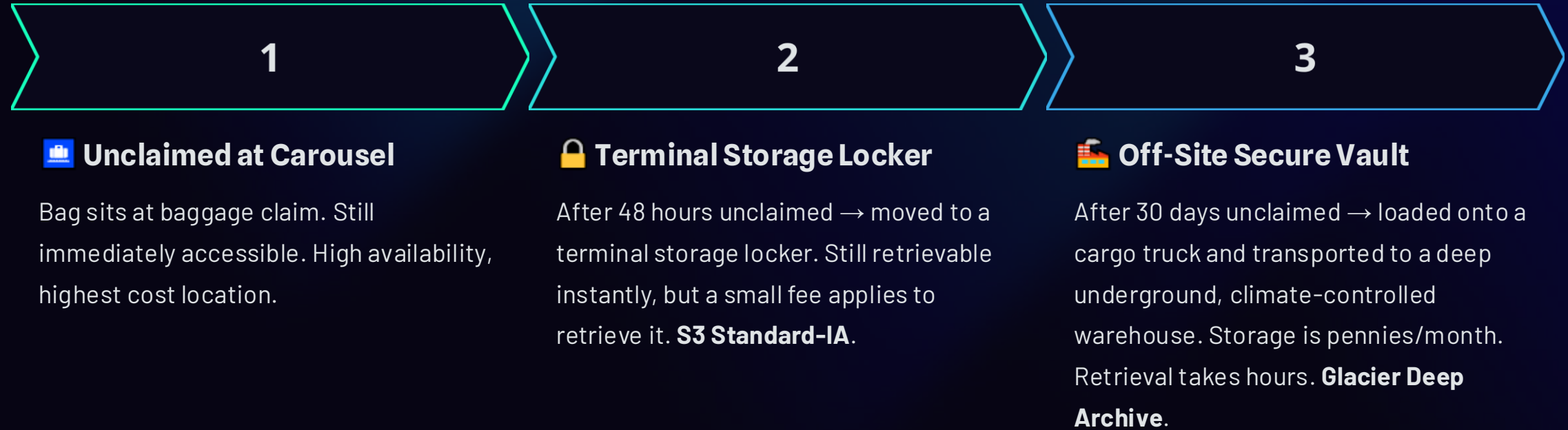
The complete raw data payload — the full file contents treated as one indivisible unit.

Sticker = Metadata

Descriptive tags attached to the object: owner, content type, creation date, classification.

Crucially: you *cannot* swap a single sock inside a bag moving on the conveyor belt. To change anything, you pull the entire suitcase out, modify it, and re-upload the complete package. This is exactly how S3 object writes work.

Airport Analogy: Lifecycle Policy & Glacier



The airport's operating procedure manual that automates all of these movements is the **S3 Lifecycle Policy** — a set of rules that triggers automatically based on age, with zero manual staff intervention required.

Airport Analogy: EFS — The Shared Digital Manifest

The Physical Scenario

Amazon EFS is like the digital flight manifest database displayed simultaneously on screens across every ticket counter, gate terminal, security checkpoint, and crew briefing room throughout the entire airport.

- Hundreds of gate agents, flight attendants, and pilots **read and write simultaneously**
- Every terminal sees the **same, consistent, real-time data**
- As more passengers board, the manifest **expands dynamically** — no technician installs a new screen

The AWS Mapping

EFS provides exactly this capability in the cloud: a **shared NFS file system** that thousands of EC2 instances across multiple Availability Zones can mount, read, and write to concurrently. As data volumes grow, EFS expands automatically to petabytes — no storage provisioning tickets required. Every server in your fleet shares the **exact same version of the data**, preventing the inconsistency issues that arise when each server has its own EBS volume.



Analogy 2: The E-Commerce Delivery Fleet

A Second Mental Model — Reinforcing the Architecture

Imagine running a regional commercial delivery service company with a large fleet of transport vehicles, warehouse facilities, and hundreds of delivery personnel operating across multiple distribution hubs. This scenario maps AWS storage services to physical fleet operations in a complementary way — especially useful for reinforcing EBS's performance-isolation tradeoff.

Delivery Fleet: All Three Services Mapped

EBS = Engine Oil Tank

An EBS volume is the physical engine oil container **bolted inside the chassis of one specific delivery truck**. It provides immediate, low-latency performance critical for that vehicle. It cannot be split with a truck on a separate highway. If the outer truck frame is retired, the oil tank can be safely unbolted and transferred to a new chassis — **the data persists**.

S3 = The Central Distribution Center

The S3 bucket is the distribution center's infinite storage yard. Items arrive as sealed shipping crates (Objects), each labeled with a tracking manifest containing source number, carrier, and expiration date (Metadata). To locate an order, you look up its exact alphanumeric shipping index code (Key). Replacing a broken item inside? Pull the entire crate, repack it, and re-seal — you **cannot surgically modify objects in place**.

EFS = The Shared Loading Dock Whiteboard

EFS is the massive whiteboard running along the wall of a shared multi-bay loading dock. Dozens of drivers and warehouse workers stand along it simultaneously, reading schedules and writing updates in real time. The board expands naturally as business grows. Everyone shares a **single, consistent version of truth** — no conflicting copies on separate boards.

CASE STUDY

MedVanguard Health Systems

A multi-state medical network generates 45 TB of patient imaging data monthly – and made every possible cloud storage mistake. Let's diagnose what went wrong and design the correct architectural remediation.



MedVanguard: Operational Context

MedVanguard Health Systems operates a multi-state network of urgent care clinics that generate approximately **45 Terabytes** of high-resolution digital X-ray imagery, patient diagnostic reports, and oncology scan files *every calendar month*. Under HIPAA compliance mandates, all patient diagnostic images must be retained for a minimum of **7 years** from the initial date of clinical evaluation.

The Data Access Pattern

Understanding how frequently data is actually accessed — versus how it is currently stored — is the core of the financial and compliance problem at MedVanguard.

Days 1–30

High-resolution images are actively accessed multiple times by attending physicians, radiologists, and consulting specialists. **Frequent, immediate access required.** S3 Standard is appropriate here.

1

Occasional Audit

If retrieved after 30 days, it is typically for an annual review or legal audit. A **retrieval delay of 3–5 hours** is entirely acceptable to clinical staff — making Glacier viable.

2

3

4

Day 31 Onward

Historical access logs show the probability of a patient record being accessed **drops by more than 96%**. Images sit idle but must remain legally accessible.

Year 7+

Legal retention window expires. Objects should be **automatically deleted** via S3 Lifecycle Expiration rules — no manual administrator action required.

The Misconfiguration: What Went Wrong

● The Problem

A prior developer configured the medical imaging application to upload all raw patient files directly into the S3 Standard storage tier – **and left them there permanently**. No Lifecycle Policy was ever configured.

Over a three-year operational window, this single misconfiguration resulted in the accumulation of **over 1.6 Petabytes** of data sitting on premium high-performance tiers, causing MedVanguard's monthly cloud storage invoice to reach **\$38,000/month**.

💣 The Attempted Fix (Made It Worse)

An IT network intern attempted to manually migrate older files by writing a download script targeting a local NAS drive in the hospital's basement. During the download loop, the local drive experienced a **hard sector crash**.

- **400 patient diagnostic files permanently degraded**
- Data integrity breach triggered
- Formal audit initiated by federal healthcare data compliance officers
- Manual migration script bypassed all cloud durability guarantees

⊗ Manual data migration scripts are never an acceptable substitute for cloud-native automation. The intern's approach bypassed 11 Nines of durability and introduced a single point of failure with zero redundancy.

The Cost Impact: By the Numbers

45TB

Generated Monthly

High-resolution X-rays, diagnostic reports, and oncology scans produced every calendar month across the clinic network.

\$38K

Monthly Storage Invoice

The unsustainable monthly bill caused by leaving petabytes of rarely accessed data in S3 Standard pricing.

1.6PB

Idle Data in S3 Standard

Three years of accumulation with no Lifecycle Policy — all sitting on the most expensive storage tier unnecessarily.

400

Files Permanently Lost

Patient diagnostic files destroyed by the intern's manual migration script when the local NAS drive suffered a sector crash.

Debate Question 1: Architectural Remediation

What storage class architecture should MedVanguard implement?

Design the correct Lifecycle Policy that satisfies both the compliance mandate and cost optimization goals. Consider the access patterns identified in the data lifecycle analysis.

Days 0–30

S3 Standard — Active clinical access. Radiologists and physicians access images multiple times. No retrieval delay acceptable. No Lifecycle transition yet.

Days 31–90

S3 Standard-IA — Access probability drops 96%. Files still accessible in milliseconds if needed, but monthly storage cost drops substantially. Retrieval fee is acceptable given rare access.

Days 91–365+

Glacier Flexible Retrieval — Annual reviews and legal audits only. 3–5 hour retrieval delay is clinically acceptable. Cost drops to a fraction of a cent per GB/month.

Day 2,555 (Year 7)

Automatic Expiration — S3 Lifecycle Expiration rule permanently deletes objects at the end of the legal retention window. Zero manual administrator action required.

Debate Question 2: Data Durability Evaluation

Manual Script vs. Amazon S3 Native Infrastructure

● Intern's Manual Script

- Single point of failure — one NAS drive, no redundancy
- No checksumming or integrity verification
- Partial transfers leave orphaned or corrupted records
- Manual execution means human error risk on every run
- No audit trail or version history for compliance officers
- Result: **400 files permanently lost, federal audit triggered**

Amazon S3 Native Architecture

- **11 Nines of durability** — data replicated across 3+ AZs automatically
- Checksumming on every PUT and GET operation
- Versioning preserves all historical states of every object
- S3 Object Lock prevents accidental or malicious deletion
- CloudTrail logs every API action — complete audit trail
- Result: **Zero data loss, continuous compliance, zero manual effort**

The concept of "11 Nines" fundamentally changes corporate liability. An organization storing HIPAA-regulated data on a local NAS drive has no structural durability guarantee — and accepts full liability for any data loss. Amazon S3 contractually guarantees durability and provides the technical documentation to satisfy federal compliance auditors.

Debate Question 3: Access Control & Data Integrity

Security Mechanisms for HIPAA -Regulated Patient Imaging Data

Medical images are among the most sensitive forms of Personally Identifiable Information (PII). Multiple security layers must be integrated with the S3 bucket to prevent unauthorized access and accidental deletion — especially during Lifecycle transitions.



S3 Default Encryption

Enable Server-Side Encryption with AWS KMS (SSE-KMS). Every object is encrypted at rest using a customer-managed KMS key. Key rotation policies enforce a 12-month automatic key refresh cycle.



Bucket Policies & IAM

Apply least-privilege IAM policies. Only the imaging application's service role should have `s3:PutObject` rights. Clinical review applications receive read-only `s3:GetObject`. Block all public access at the account level.



S3 Versioning & Object Lock

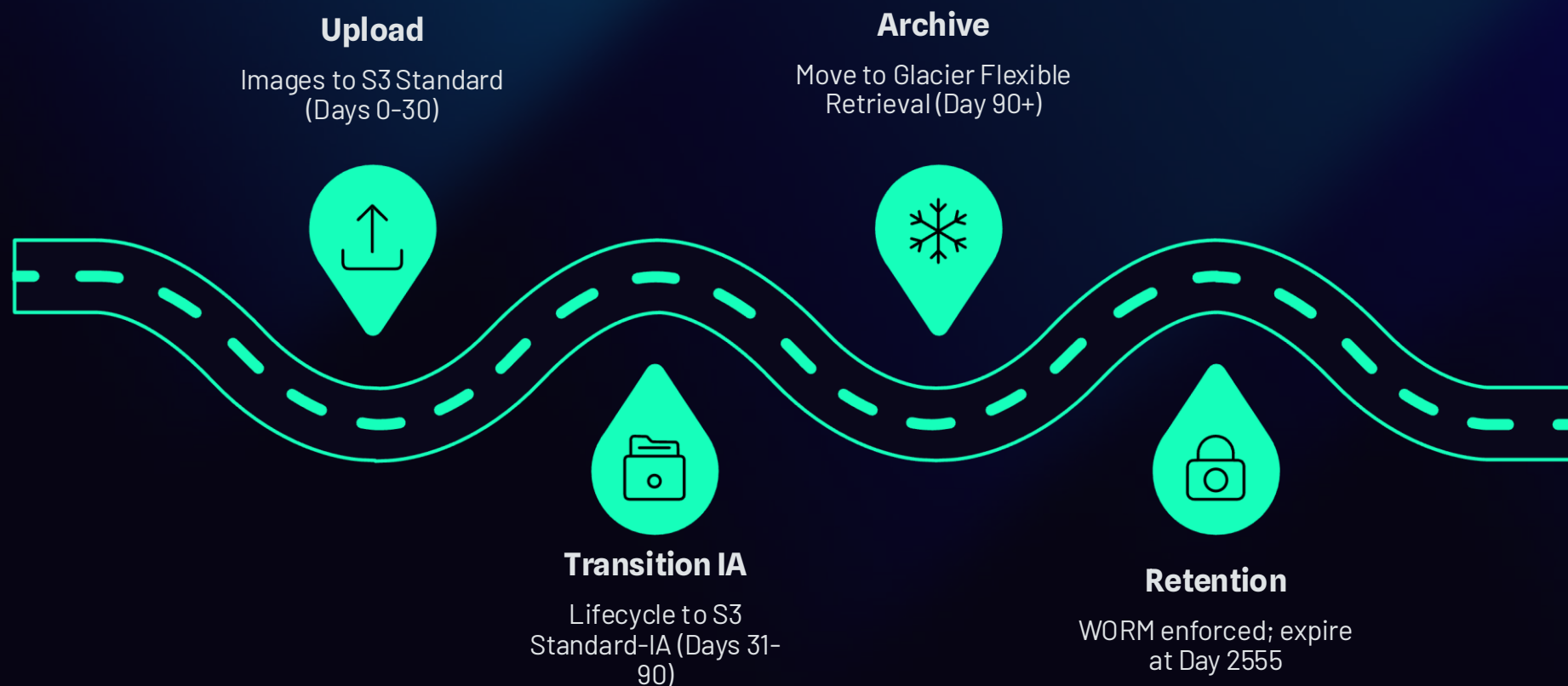
Enable S3 Versioning to preserve all historical versions of every patient image. Enable S3 Object Lock in **Compliance Mode** with a 7-year WORM (Write Once Read Many) retention period — preventing deletion by any user, including AWS root accounts.



CloudTrail & Access Logging

Enable AWS CloudTrail data events on the S3 bucket and S3 Server Access Logging. Every API call — who accessed which object, at what time, from which IP — is logged to a separate, write-protected audit bucket for federal compliance review.

MedVanguard: The Correct Architecture



This architecture eliminates manual administrator intervention entirely, enforces HIPAA data retention through S3 Object Lock WORM policies, and is projected to reduce MedVanguard's monthly cloud storage invoice from **\$38,000 to approximately \$3,200** – a cost reduction exceeding 90% – while increasing data durability from an unreliable local NAS to 11 Nines of AWS-guaranteed structural reliability.

Key Takeaways: Storage Service Decision Matrix

Use this decision framework when evaluating storage requirements in any AWS architecture engagement. The right service is determined by access pattern, concurrency requirement, and cost tolerance — not familiarity or habit.

Requirement	Correct Service	Why
Run a virtual machine OS or database	Amazon EBS (gp3)	Block-level access, low latency, single-instance attachment
10 web servers share the same upload folder	Amazon EFS	NFS protocol, concurrent multi-instance read/write access
Host static website assets at global scale	Amazon S3 Standard	Object storage, global HTTPS access, infinite scalability
Store monthly reports accessed a few times/year	S3 Standard-IA	Lower storage cost; retrieval fee acceptable for infrequent access
7-year HIPAA compliance archive	S3 Glacier Deep Archive	Lowest cost, WORM via Object Lock, 11 Nines durability
Automate tier movement as data ages	S3 Lifecycle Policy	JSON rules trigger automatic transitions with zero manual effort

Day 7 Summary: Core Concepts Reviewed



Block vs. Object

Block storage (EBS) modifies specific sectors. Object storage (S3) replaces entire file payloads. Choose based on mutation frequency and access pattern.



Amazon S3

Globally scalable object storage. Objects = Key + Value + Metadata. 11 Nines durability. Multiple storage class tiers from Standard to Glacier Deep Archive.



Amazon EBS

High-performance block volumes for single EC2 instances within one Availability Zone. Persists independently from instance lifecycle. Use Snapshots for backup and migration.



Amazon EFS

Managed NFS file system for concurrent multi-instance access across multiple AZs. Scales automatically to petabytes. No storage provisioning required.



S3 Lifecycle Policies

JSON rules that automatically transition objects between tiers as they age. The primary tool for cloud cost optimization at scale — eliminate manual migration scripts permanently.